

Navigators Technical Documentation

Navigators — AI/ML Neural Pipeline Specification

Document Identifier: ML-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

1. Machine Learning Task Formulation

The primary task of the AI pipeline in Navigators is **Inertial Velocity Regression**: Given a temporal window of normalized tri-axial specific force and angular velocity measurements captured by a smartphone mounted inside a vehicle, predict the instantaneous two-dimensional horizontal velocity vector $[v_N, v_E]$ in the local navigation frame:

$$f_{\theta} : \mathbb{R}^{W \times 6} \rightarrow \mathbb{R}^2$$

Where: - $W = 200$: Window size (200 samples at 10 Hz = 20.0 s temporal horizon). - Input channels ($C = 6$): $[\mathbf{a}_x^V, \mathbf{a}_y^V, \mathbf{a}_z^V, \boldsymbol{\omega}_x^V, \boldsymbol{\omega}_y^V, \boldsymbol{\omega}_z^V]$. - Target labels: $[v_N, v_E]$ in meters per second (m/s).

2. Model Architecture: Temporal Convolutional Network (TCN)

Vehicle inertial dynamics exhibit non-linear acceleration patterns and long-range temporal dependencies. Rather than recurrent architectures (RNN/LSTM) that suffer from sequential execution bottlenecks and exploding/vanishing gradients, Navigators deploys a **Temporal Convolutional Network (TCN)** implemented in `src/models/tcn_model.py`.

```

Input: Tensor (Batch, 6 Channels, 200 Timesteps)
|
├─> Block 1: CausalConv1d(6 -> 64, kernel=7, dilation=1) + BN + ReLU + Dropout(0.2)
|           CausalConv1d(64 -> 64, kernel=7, dilation=1) + BN + ReLU + Dropout(0.2) + Skip(1x1)
|
├─> Block 2: CausalConv1d(64 -> 64, kernel=7, dilation=2) + BN + ReLU + Dropout(0.2)
|           CausalConv1d(64 -> 64, kernel=7, dilation=2) + BN + ReLU + Dropout(0.2) + Identity
|
├─> Block 3: CausalConv1d(64 -> 128, kernel=7, dilation=4) + BN + ReLU + Dropout(0.2)
|           CausalConv1d(128 -> 128, kernel=7, dilation=4) + BN + ReLU + Dropout(0.2) + Skip(1x1)
|
├─> Block 4: CausalConv1d(128 -> 128, kernel=7, dilation=8) + BN + ReLU + Dropout(0.2)
|           CausalConv1d(128 -> 128, kernel=7, dilation=8) + BN + ReLU + Dropout(0.2) + Identity
|
└─>
      Global Average Pooling across Time Dimension (128 Channels)
      |
      └─>
            Linear Projection Head: Linear(128 -> 64) -> ReLU -> Dropout(0.2) -> Linear(64 -> 2)
            |
            └─>
                  Output: Velocity Tensor (Batch, 2) -> [v_North, v_East]

```

2.1 Mathematical Properties of the Architecture

1. **Strict Causality:** Convolutions employ asymmetric left-padding:

$$\text{pad}_{\text{left}} = (K - 1) \cdot d$$

where kernel size $K = 7$ and dilation $d \in \{1, 2, 4, 8\}$. Convolutions never access future timesteps $t' > t$, eliminating acausal lookahead leakage.

2. **Receptive Field:** The receptive field RF of a L -layer dilated convolutional network is:

$$RF = 1 + \sum_{l=0}^{L-1} 2(K - 1) \cdot 2^l = 1 + 2(7 - 1)(1 + 2 + 4 + 8) = 1 + 12(15) = 181 \text{ samples}$$

This covers virtually the entire 200-sample (18.1 s) window.

3. **Parameter Footprint:** The production TCN model contains **5,400,322 parameters** (~21.6 MB unquantized float32, ~6.8 MB ONNX runtime package).

3. Data Preprocessing & Leakage Prevention

3.1 Session-Level Splitting (Zero Temporal Leakage)

A critical flaw in naive time-series modeling is shuffling sliding windows across train, validation, and test splits. Adjacent 200-sample windows with 50-sample stride overlap by 75%, causing catastrophic data leakage and artificially inflated accuracy.

Navigators strictly enforces **Session-Level Splitting**: - Each driving recording file (e.g., in IO-VNBD) is treated as an indivisible monolithic session. - 100 complete sessions → Training Set (63,423 windows). - 22

complete sessions → Validation Set (30,349 windows). - 22 complete sessions → Test Set (11,934 windows). - No temporal overlap exists between train, validation, and test splits.

3.2 Z-Score Normalization

Normalization statistics are computed strictly over the training sessions:

```
{
  "features": ["ACCELEROMETER X", "ACCELEROMETER Y", "ACCELEROMETER Z", "GYROSCOPE X",
    "GYROSCOPE Y", "GYROSCOPE Z"],
  "mean": [-0.023565, -0.035606, 8.931498, -0.000460, -0.000460, -0.000024],
  "std": [1.599169, 1.517184, 1.158331, 0.113666, 0.113666, 0.131972],
  "method": "z-score"
}
```

4. Training Hyperparameters & Optimization

Hyperparameter	Value	Description
Optimizer	AdamW	Decoupled weight decay regularization.
Learning Rate	1×10^{-3}	Initial base learning rate.
Weight Decay	1×10^{-4}	L_2 penalty preventing weight explosion.
Batch Size	64 (train_and_evaluate: 256)	Mini-batch sample size.
Loss Function	Combined MSE + Directional Loss	Penalizes speed error and velocity vector orientation error.
Directional Weight	0.3	Weight assigned to unit heading error $\mathcal{L}_{ang} = 1 - \cos(\theta, \hat{\theta})$.
Early Stopping	15 Epochs	Stops training if validation loss fails to improve for 15 epochs.
LR Scheduler	Cosine Annealing	Decays learning rate smoothly to 10^{-6} over training horizon.

5. Edge Export & ONNX Verification

To transition from server training to client-side browser execution: 1. **Export Pipeline**

(`src/edge/onnx_export.py`): Exports PyTorch checkpoint to ONNX with dynamic batch dimension: `python torch.onnx.export( model, dummy_input, "simulator/model.onnx", input_names=["imu_window"], output_names=["velocity"], dynamic_axes={"imu_window": {0: "batch"}, "velocity": {0: "batch"}}, opset_version=17 )` 2. **Parity Validation (`scripts/verify_onnx.py`)**: Asserts that ONNX Runtime

predictions match PyTorch outputs on identical input windows within strict tolerances: - Absolute Tolerance (atol): 1×10^{-5} - Relative Tolerance (rtol): 1×10^{-4}

Developed by Navigators